



Construire des agents IA fiables

De l'alerte au dossier d'incident, avec Python et Gemini.

Vous allez construire une boucle, puis la tester.

- | | | |
|-----------|---|---------------|
| 01 | Comprendre les outils et la boucle
Un exemple suivi depuis l'alerte jusqu'au premier résultat. | 10 min |
| 02 | Construire dans le notebook
Ouvrir le notebook, puis compléter les checkpoints 1 à 3. | 31 min |
| 03 | Faire échouer l'agent et vérifier les protections
Checkpoint 4 : comparer les résultats aux comportements attendus. | 5 min |
| 04 | Emporter le dossier et expliquer son travail
Retrouver les preuves et proposer un test pour son propre métier. | 4 min |

Quelles tâches confiez-vous aujourd'hui à une IA ?

Vos premiers usages

Rédiger un texte, expliquer du code, reformuler une idée... Quel était votre premier usage ?

Vos usages actuels

Quelles tâches lui confiez-vous maintenant ? Pour les réaliser, de quelles informations et de quels outils a-t-elle besoin ?

Pour examiner un incident en cours, le système doit pouvoir consulter son état actuel.

09:42 : une clinique signale une hausse de température.

Alerte CC-204

Clinique KCARE-ADJ-01

12,4 °C depuis 52 min

Vaccins · lot VX-204

Le réfrigérateur de la clinique a déclenché une alerte. Votre équipe doit vérifier les mesures et retrouver la procédure à suivre.

Vous êtes l'équipe de garde.

Construisez un agent qui prépare le dossier d'incident et demande la décision d'un opérateur.

Exercice fictif : données, procédures et opérateur simulés. Aucun acte médical ni aucune action physique.

Qui fait quoi quand on traite cette alerte ?

Le LLM propose

Un LLM est un modèle de langage. Il utilise son contexte pour produire du texte ou un appel d'outil. Ici, il demande la mesure.

L'application exécute

Notre code Python fournit les outils au modèle, vérifie les arguments et appelle la fonction qui lit la mesure.

L'agent poursuit la tâche

L'agent est l'application qui poursuit l'objectif : le modèle reçoit le résultat de l'outil et choisit la suite, jusqu'à une réponse ou un arrêt.

Dans un workflow fixe, le code impose les étapes. Ici, le modèle en choisit certaines selon les résultats.

Le modèle demande une mesure. Python la lit.

1. Proposition du modèle

```
{  
  "name": "get_clinic_status",  
  "arguments": {  
    "clinic_id": "KCARE-ADJ-01"  
  }  
}
```

Le nom choisit la fonction ; les arguments précisent la clinique. Python les valide avant l'exécution.

2. Résultat renvoyé par l'outil

```
{  
  "clinic_id": "KCARE-ADJ-01",  
  "temperature_c": 12.4,  
  "excursion_minutes": 52,  
  "sensor_status": "OK"  
}
```

Cet extrait contient la mesure lue. On conserve la sortie réelle, ainsi que les erreurs éventuelles.

Appel normalisé et résultat abrégé du scénario simulé. Les schémas des outils sont fournis au modèle.

Chaque résultat prépare la décision suivante.

À chaque tour, le modèle propose un appel d'outil ou une réponse.



Le modèle reçoit cette observation au tour suivant.

Si le modèle propose une réponse : Python vérifie les preuves, puis la rend ou la bloque.

La mesure est connue. Quelle information manque encore ?

Moment	Ce qui circule	Ce que cela permet
Tour 1	get_clinic_status clinic_id : KCARE-ADJ-01	Lire l'état de la clinique.
Observation	12,4 °C · 52 min · capteur OK	Conserver les faits dans l'historique.
Tour 2	search_cold_chain_sop query : température excursion critique	Retrouver la procédure adaptée.

Sans retour du résultat, le modèle ne sait pas ce que le premier outil a trouvé.

Cinq fonctions pour examiner l'alerte.

Question	Outil Python	Résultat attendu
Que mesure le capteur ?	<code>get_clinic_status</code>	La télémétrie de la clinique.
Quelle procédure appliquer ?	<code>search_cold_chain_sop</code>	La procédure locale retrouvée.
Que disent les règles du lab ?	<code>assess_excursion_risk</code>	La gravité et l'action proposée.
Comment documenter l'alerte ?	<code>create_incident</code>	Un identifiant de dossier.
L'opérateur approuve-t-il ?	<code>request_human_review</code>	Une décision liée à cet incident.

Le chemin dépend de la demande. Les règles d'autorisation restent dans le code.

Trois erreurs que notre code doit reconnaître.

La mesure change

L'outil lit 12,4 °C. Le modèle transmet ensuite 5 °C au calcul du risque. Le code doit comparer cette valeur à la mesure lue.

L'appel se répète

Le modèle redemande exactement la même mesure. Le code détecte l'appel déjà exécuté et arrête la boucle.

L'accord manque

Le modèle veut conclure avant la revue requise. Le code exige une approbation pour le bon incident et la bonne action.

Dans le notebook, vous allez provoquer ces erreurs et vérifier où le système s'arrête.

Passer de l'alerte à la première mesure.

1

Prédisez le premier appel.

Quelle fonction permet de connaître l'état de KCARE-ADJ-01 ?

2

Complétez TODO 1-2.

Donnez au modèle l'historique et les schémas, puis récupérez son appel.

3

Comparez avec votre prédiction.

Vérifiez le nom de l'outil et la clinique demandée.

Vous avez réussi si l'appel est `get_clinic_status` avec `clinic_id = KCARE-ADJ-01`.

Prévisualisation déterministe en mode mock. Le notebook explique le passage à Gemini.

Renvoyer au modèle ce que l'outil a trouvé.

1

Exécutez et tracez l'appel : TODO 3–4.

Conservez l'outil, ses arguments, le résultat et son statut.

2

Complétez l'historique : TODO 5–6.

Ajoutez la proposition du modèle, puis le résultat renvoyé par l'outil.

3

Lisez la première entrée de trace.

Retrouvez la température obtenue et l'appel qui l'a produite.

Le prochain tour doit recevoir la mesure réellement lue. La trace permet de la retrouver.

Autoriser la suite ou arrêter la mission.

- 1** **Contrôlez les preuves : TODO 7–8.**
La mesure doit être cohérente ; l'accord doit viser le bon incident et la bonne action.
- 2** **Bloquez les appels répétés : TODO 9.**
Un appel identique déjà exécuté doit entraîner un arrêt contrôlé.
- 3** **Lancez la mission, puis un contre-exemple.**
Retrouvez dans la trace ce qui autorise la conclusion ou impose l'arrêt.

Sur la mission critique, les cinq outils sont tracés et l'approbation de l'opérateur simulé est enregistrée.

Le modèle dit « résolu ». Peut-on accepter sa réponse ?

La réponse proposée

« Tout est sûr ; l'incident est résolu. »

Le modèle vient de lire la mesure, la procédure et le niveau de risque.

Ce que montre la trace

Le risque exige une revue.

Aucun dossier créé.

Aucune approbation enregistrée.

Python bloque la conclusion : `stopped / review_required`. Il manque l'accord requis.

Réponse volontairement fautive du simulateur : « Everything is safe; incident resolved. »

Un test passe quand le comportement attendu est respecté.

Scénario	Résultat attendu	Solution	Test
Alerte critique complète	human_approved	human_approved	PASS
Aucune investigation	blocked	blocked	PASS
Approbation absente	review_required	review_required	PASS
Appel identique répété	blocked	blocked	PASS

L'arrêt peut être le résultat correct : les tests vérifient aussi les refus attendus.

Extraits de la solution : dix scénarios déterministes, dont sept erreurs volontaires. Aucun appel API.

Vérifier chaque protection sur un cas précis.

1

Complétez TODO 10.

Un scénario passe seulement si toutes ses vérifications sont vraies.

2

Lancez les dix scénarios.

Comparez le chemin normal et les erreurs provoquées avec les résultats attendus.

3

Expliquez un résultat à votre binôme.

Quelle erreur avez-vous provoquée ? Quel contrôle l'a détectée ?

Après la mission approuvée et 10/10 aux évaluations, le dossier JSON devient téléchargeable.

Les cinq minutes incluent la lecture des résultats. Les évaluations restent en mode déterministe.

Le test doit détecter une protection retirée.

Sans le contrôle anti-répétition

Le modèle peut redemander le même outil.
Le cas repeat ne respecte plus le comportement attendu : son test échoue.

Avec le contrôle rétabli

Python reconnaît l'appel déjà exécuté et arrête la boucle. Le même test passe parce que cette répétition est bloquée.

10/10 couvre ces dix scénarios. Un nouveau risque demande un nouveau cas de test.

Un dossier que vous pouvez expliquer et vérifier.

INCIDENT

INC-001

APPELS D'OUTILS

5

OPÉRATEUR SIMULÉ

APPROVED

ÉVALUATIONS

10 / 10

```
{
  "alert_id": "CC-204",
  "execution_evidence": [{
    "tool": "get_clinic_status",
    "status": "success"
  }]
}
```

Le fichier contient les mesures, les appels et leurs résultats, la décision simulée et les évaluations. Votre contre-exemple y est aussi ajouté.

Choisissez une décision du dossier et retrouvez les éléments qui la justifie.

Extrait abrégé du dossier de la solution en mode mock. Le fichier documente l'exercice ; aucune intervention physique.

POUR CONTINUER

Reprenez le code et ajoutez votre propre scénario.

[Dépôt de l'atelier · notebooks français et anglais](#)

Ouvrir Colab, retrouver la solution et télécharger les supports.

[Anthropic · Building effective agents](#)

Lecture complémentaire sur les workflows et les agents.

[Google · Gemini API rate limits](#)

Consulter les limites avant de prévoir plusieurs appels en parallèle.

MERCI

Quelle règle testeriez-vous dans votre métier ?

Nommez une action, puis le cas où votre agent devrait s'arrêter.

Exemple : préparer un remboursement, puis bloquer son exécution si l'accord requis manque.



[Code et notebooks](#)

[Scannez pour ouvrir le dépôt](#)

BOSSA Chabel · Ingénieur logiciel · ENSGMM

github.com/chabelbossa · linkedin.com/in/chabel-holdo-bossa